2026/10/29

Sistema de Impresoras en Red

XAVIER SANTAMARIA Y MATTHEWS VEGA

Sistemas Operativos

2026

Contenido

Descripción 2
Análisis 2
Conclusión del análisis 3
Planificación y Diseño 4
Recurso compartido 4
Representación del recurso en Python 4
Diseño de clases 5
Sección crítica 8
Comparación de las representaciones 10
Bibliografía 11

Descripción

El Sistema de Impresoras en Red permite que varios usuarios envíen trabajos de impresión desde diferentes equipos conectados a una misma red. El problema consiste en coordinar las solicitudes que pueden llegar simultáneamente, organizar los trabajos pendientes y asignarlos a impresoras disponibles, evitando conflictos y manteniendo el estado de los recursos.

Análisis

Las unidades de ejecución participantes son los procesos o hilos que reciben y gestionan las solicitudes de impresión, junto con los procesos o hilos que administran las impresoras. Los recursos compartidos principales son las impresoras disponibles y la cola de trabajos pendientes. Se requiere cooperación porque varias solicitudes pueden llegar al mismo tiempo; por ello, el sistema debe ordenar los trabajos, controlar la disponibilidad de las impresoras y evitar que dos unidades asignen simultáneamente un mismo recurso. La siguiente tabla compara cómo se implementaría cada modelo en esta situación.

Tabla 1 Comparación de modelos en el Sistema de Impresoras en Red

Criterio	Memoria Compartida	Paso de Mensajes
Ubicación del recurso	La cola de trabajos y el estado de cada impresora (libre/ocupada) se guardan en una zona de memoria a la que acceden todos los procesos.	Un proceso **servidor de impresión** (spooler) es el único que guarda la cola y el estado de las impresoras.
Envío de un documento	El usuario escribe directamente su trabajo al final de la cola compartida.	El usuario envía un mensaje y el servidor lo agrega a su cola.
Asignación de impresora	Cada impresora libre revisa la cola compartida y toma el siguiente trabajo.	El servidor elige el siguiente trabajo y envía un mensaje a una impresora libre. La impresora avisa con un mensaje cuando termina.
Control de conflictos	Se necesitan mutex para proteger la cola y semáforos para contar las impresoras libres y los trabajos pendientes. Un error en la sincronización provoca condiciones de carrera.	Los conflictos se evitan por diseño, porque solo el servidor modifica la cola. Los mensajes se atienden uno por uno.
Orden de atención	Depende de que todos respeten correctamente el acceso a la cola (FIFO protegida).	El servidor garantiza el orden FIFO o por prioridad de forma centralizada.
Adecuación a una red	Poco natural, porque los usuarios están en equipos distintos que no comparten físicamente la memoria.	Natural, porque la comunicación en red ya funciona mediante mensajes.
Ventajas	Acceso muy rápido a los datos y menor costo de comunicación dentro de una misma máquina.	Mayor seguridad y orden, fácil de escalar (agregar usuarios o impresoras) y menos errores de sincronización.

Criterio	Memoria Compartida	Paso de Mensajes
Desventajas	Mayor complejidad para sincronizar, riesgo de interbloqueos y datos corruptos. Difícil de aplicar entre equipos distintos.	Más lento por el envío de mensajes. El servidor puede convertirse en un cuello de botella o punto único de falla.

Conclusión del análisis

Tras la comparación, el modelo más adecuado para este escenario es el de Paso de Mensajes. Los usuarios y las impresoras están distribuidos en una red y no comparten físicamente una misma memoria, por lo que comunicarse mediante mensajes refleja de forma directa el funcionamiento real del sistema. Además, centralizar la cola en un servidor de impresión garantiza que cada trabajo se asigne a una sola impresora, que se respete el orden de llegada y que ningún documento se pierda o se mezcle, sin depender de mecanismos de sincronización complejos. Aunque la Memoria Compartida ofrece mayor velocidad, sus ventajas pesan menos en este caso, porque el tiempo de impresión es mucho mayor que el de enviar un mensaje. En cambio, sus riesgos de condiciones de carrera e interbloqueos sí afectarían directamente la fiabilidad del sistema.

Planificación y Diseño

En esta fase se define cómo se construirá la solución a partir del modelo de Paso de Mensajes elegido en el análisis. Se identifica el recurso compartido, se proponen dos formas de representarlo en Python, se diseñan las clases y los hilos que participan, y se determina la sección crítica junto con la forma en que el servidor procesa las solicitudes para mantener el recurso consistente.

Recurso compartido

El recurso compartido del sistema es el **estado del servicio de impresión**, formado por dos elementos que siempre cambian juntos:

- **El conjunto de impresoras de la red**, con el estado de cada una (libre u ocupada) y el trabajo que está imprimiendo.
- **La cola de trabajos pendientes**, donde esperan, en orden de llegada, los documentos que todavía no tienen una impresora asignada.

En la situación real, este recurso representa la información que necesita el administrador de impresión para decidir qué documento se imprime a continuación y en qué impresora. Cada vez que un usuario envía un documento, la cola crece; cuando una impresora recibe un trabajo, la cola se reduce y la impresora pasa a estar ocupada; y cuando termina, vuelve a quedar libre.

Requiere coordinación porque varios usuarios pueden enviar documentos al mismo tiempo y las impresoras terminan sus trabajos en momentos distintos. Si cada unidad de ejecución modificara el recurso por su cuenta, una impresora podría recibir dos trabajos a la vez, un documento podría imprimirse dos veces o perderse, y no se respetaría el orden de llegada. Por eso, siguiendo el modelo de Paso de Mensajes, el recurso se guarda únicamente dentro del **servidor de impresión (spooler)**: los usuarios y las impresoras no lo modifican directamente, sino que envían mensajes, y el servidor es el único que lo consulta y actualiza.

Representación del recurso en Python

Se proponen dos formas de representar el recurso. Para que puedan compararse, ambos ejemplos describen el mismo momento del sistema: tres impresoras ocupadas y dos trabajos en espera.

Tipo básico: contadores enteros

El estado del sistema se resume con dos números enteros: la cantidad de impresoras libres y la cantidad de trabajos pendientes.

```
# 3 impresoras ocupadas y 2 trabajos en espera
impresoras_libres = 0
trabajos_pendientes = 2

# Llega una nueva solicitud
trabajos_pendientes += 1

# Una impresora termina y el servidor le asigna otro trabajo
impresoras_libres += 1
if impresoras_libres > 0 and trabajos_pendientes > 0:
    impresoras_libres -= 1
    trabajos_pendientes -= 1
```

Es la representación más simple y ocupa muy poca memoria, pero solo indica *cuántas* impresoras están libres y *cuántos* trabajos esperan. No permite saber cuál impresora está libre, qué documento imprime cada una ni en qué orden llegaron las solicitudes.

Tipo complejo: diccionario y cola

Las impresoras se guardan en un diccionario (`dict`) cuya clave es el nombre de la impresora y cuyo valor registra su estado y el trabajo asignado. Los trabajos pendientes se guardan en una `deque`, la cola de doble extremo del módulo `collections` (Python Software Foundation, n.d.-a), donde cada trabajo es a su vez un diccionario con sus datos.

```
from collections import deque

# 3 impresoras ocupadas y 2 trabajos en espera
impresoras = {
    "IMP-1": {"estado": "ocupada", "trabajo": 2},
    "IMP-2": {"estado": "ocupada", "trabajo": 3},
    "IMP-3": {"estado": "ocupada", "trabajo": 4},
}
cola_trabajos = deque([
    {"id": 5, "usuario": "Ana",  "documento": "informe.pdf", "paginas": 3},
    {"id": 6, "usuario": "Luis", "documento": "tarea.docx",  "paginas": 8},
])

# Llega una nueva solicitud: se agrega al final de la cola
cola_trabajos.append(trabajo_nuevo)

# IMP-2 termina: recibe el primer trabajo de la cola (FIFO)
siguiente = cola_trabajos.popleft()
impresoras["IMP-2"] = {"estado": "ocupada", "trabajo": siguiente["id"]}
```

Esta representación registra cuáles impresoras están libres, qué trabajo imprime cada una y el orden exacto de los trabajos en espera. La cantidad de impresoras libres no se pierde: se puede calcular contando las impresoras con estado "libre".

Diseño de clases

La solución se organiza en tres clases. Las tres heredan de `threading.Thread`, porque cada objeto representa una unidad de ejecución que trabaja en paralelo con las demás (Python Software Foundation, n.d.-c). Las clases no comparten las variables del recurso: la comunicación se hace únicamente depositando mensajes en **buzones** implementados con `queue.Queue`. El servidor y cada impresora tienen su propio buzón; los usuarios solo conocen el buzón del servidor. La Tabla 2 resume cada clase y la Figura 1 muestra el diagrama de clases.

Tabla 2 Clases de la solución

ServidorImpresion (1 hilo, hereda de `threading.Thread`)	
Responsabilidad	Es el servidor de impresión (spooler) y el único dueño del recurso. Recibe las solicitudes, ordena los trabajos, los asigna a impresoras libres y las libera cuando terminan.
Atributos relacionados con el recurso	`buzon`: Queue donde llegan todos los mensajes. `impresoras`: dict con el estado de cada impresora (recurso compartido). `cola_trabajos`: deque con los trabajos pendientes (recurso compartido). `buzones_impresoras`: dict con el buzón de cada impresora. `activo`: mantiene el bucle en ejecución. `cerrando`: indica que ya no llegarán más trabajos.
Métodos que consultan o modifican el recurso	`run()`: toma un mensaje a la vez del buzón y lo procesa. `recibir_trabajo(trabajo)`: agrega el trabajo al final de la cola. `asignar_trabajos()`: mientras haya trabajos e impresoras libres, saca el primero de la cola, marca la impresora como ocupada y le envía el trabajo. `liberar_impresora(nombre)`: marca la impresora como libre. `preparar_cierre()`: si se recibió FIN, la cola está vacía y todas las impresoras están libres, envía FIN a cada impresora y detiene el bucle. `mostrar_estado()`: consulta el recurso y muestra la cola y el estado de cada impresora.
Usuario (N hilos, hereda de `threading.Thread`)	
Responsabilidad	Simula a una persona que, desde un equipo de la red, envía documentos para imprimir.
Atributos relacionados con el recurso	`nombre`: identifica al usuario. `documentos`: lista de documentos por enviar. `buzon_servidor`: referencia al buzón del servidor. No tiene acceso a la cola ni a las impresoras.
Métodos que consultan o modifican el recurso	`run()`: recorre sus documentos y los envía con pausas aleatorias. `enviar_trabajo(documento, paginas)`: arma el trabajo y deposita el mensaje IMPRIMIR en el buzón del servidor. No modifica el recurso directamente.
Impresora (M hilos, hereda de `threading.Thread`)	
Responsabilidad	Simula una impresora física: espera un trabajo, lo imprime y avisa al servidor cuando termina.
Atributos relacionados con el recurso	`nombre`: identifica a la impresora (IMP-1, IMP-2, ...). `buzon`: Queue propia donde el servidor deja los trabajos. `buzon_servidor`: referencia al buzón del servidor.

Métodos que consultan o modifican el recurso	`run()`: espera mensajes en su buzón. `imprimir(trabajo)`: simula la impresión con una pausa proporcional al número de páginas. `avisar_fin(trabajo)`: deposita el mensaje TERMINADO en el buzón del servidor.

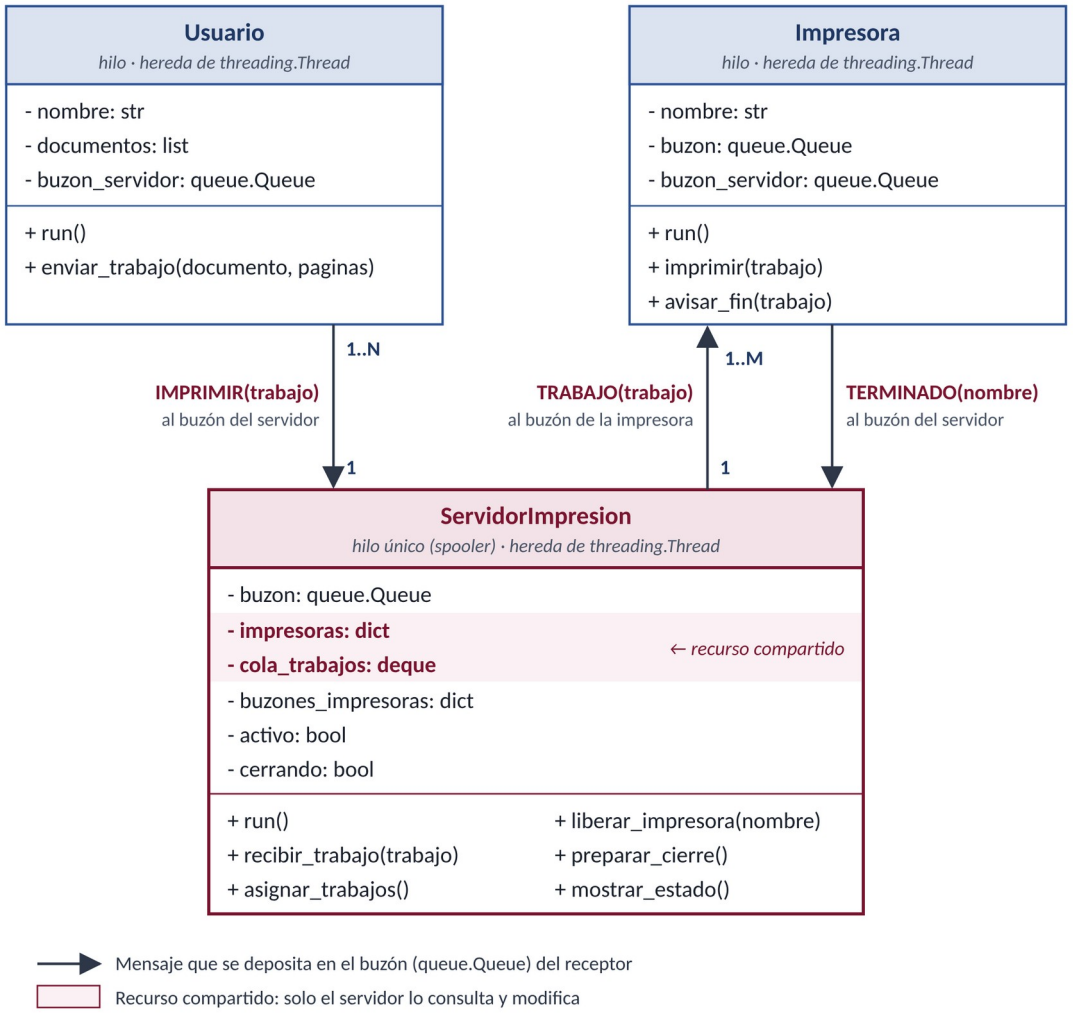


Figura 1 Diagrama de clases del sistema

Relación entre hilos y clases

Cada objeto de las clases `Usuario` e `Impresora` se ejecuta como un hilo independiente, y existe un único hilo de la clase `ServidorImpresion`. El programa principal crea primero el servidor, luego las impresoras (registrando el buzón de cada una en el servidor) y por último los usuarios, y los inicia con `start()`. Cuando todos los usuarios terminan, el programa principal envía el mensaje FIN al servidor.

Los hilos se relacionan solo a través de mensajes: los usuarios escriben en el buzón del servidor, el servidor escribe en el buzón de cada impresora y las impresoras responden en el buzón del

servidor. Cada mensaje es una tupla `(tipo, datos)`. La Tabla 3 describe los mensajes del sistema.

Tabla 3 Mensajes que intercambian los hilos

Mensaje	Emisor	Receptor	Contenido	Acción al recibirlo
IMPRIMIR	Usuario	Servidor	Datos del trabajo: id, usuario, documento y páginas.	El servidor agrega el trabajo al final de la cola e intenta asignarlo.
TRABAJO	Servidor	Impresora	Trabajo asignado.	La impresora lo imprime y, al terminar, envía TERMINADO.
TERMINADO	Impresora	Servidor	Nombre de la impresora.	El servidor marca la impresora como libre y le asigna el siguiente trabajo de la cola, si existe.
FIN	Programa principal	Servidor	Sin datos.	El servidor deja de aceptar trabajos y espera a que la cola quede vacía y todas las impresoras estén libres.
FIN	Servidor	Impresora	Sin datos.	La impresora termina su hilo.

Sección crítica

Aunque en el modelo de Paso de Mensajes los hilos no comparten las variables del recurso, sí existe una sección crítica: el conjunto de operaciones que modifican la cola de trabajos y el estado de las impresoras.

Operaciones que deben coordinarse:

1. **Agregar un trabajo** a la cola cuando llega una solicitud de impresión.
2. **Asignar un trabajo**: comprobar que hay una impresora libre, marcarla como ocupada y sacar el primer trabajo de la cola. Estas tres acciones forman una sola operación; si se separan, el estado puede cambiar entre la comprobación y la asignación.
3. **Liberar una impresora** cuando avisa que terminó.

Problemas si se ejecutaran simultáneamente:

- **Condición de carrera en la asignación:** dos hilos consultan el estado al mismo tiempo, ambos ven que IMP-1 está libre y ambos le asignan un trabajo. Uno de los documentos se pierde o las impresiones se mezclan.
- **Trabajo duplicado:** dos impresoras leen el primer trabajo de la cola antes de que alguna lo retire, y el mismo documento se imprime dos veces.
- **Actualización perdida:** si dos hilos actualizan al mismo tiempo la cola o un contador (leer, modificar y escribir), uno de los cambios se pierde y el sistema queda con datos que no coinciden con la realidad.

- **Orden incorrecto:** los trabajos dejan de atenderse en el orden en que llegaron.

Mecanismo propuesto: procesamiento de mensajes en el servidor.

Como se eligió el modelo de Paso de Mensajes, la consistencia del recurso no depende de que cada hilo use un mutex, sino de la forma en que el servidor procesa las solicitudes:

- **Un solo dueño del recurso.** La cola y el diccionario de impresoras son atributos del `ServidorImpresion` y ningún otro hilo tiene referencia a ellos. Incluso la consulta del estado (`mostrar_estado()`) se hace desde el propio servidor.
- **Buzón seguro entre hilos.** Todas las solicitudes llegan al buzón del servidor, un objeto `queue.Queue`. Esta clase ya incluye la sincronización necesaria, porque internamente usa un candado (Lock) y variables de condición (Python Software Foundation, n.d.-b). Así, varios usuarios pueden depositar mensajes al mismo tiempo sin que se pierdan, y estos se conservan en orden de llegada.
- **Un mensaje a la vez.** El método `run()` del servidor toma un mensaje con `get()`, lo procesa por completo y solo entonces toma el siguiente. De esta forma, las operaciones de la sección crítica nunca se entrelazan.
- **Las impresoras no toman trabajos por su cuenta.** Cada impresora espera en su propio buzón a que el servidor le envíe un trabajo, por lo que es imposible que dos impresoras tomen el mismo.

El siguiente fragmento muestra el bucle del servidor previsto para la implementación:

```
def run(self):
    while self.activo:
        tipo, datos = self.buzon.get()     # espera y toma UN mensaje
        if tipo == "IMPRIMIR":
            self.recibir_trabajo(datos)    # lo agrega a la cola
        elif tipo == "TERMINADO":
            self.liberar_impresora(datos)  # la impresora queda libre
        elif tipo == "FIN":
            self.cerrando = True           # no llegarán más trabajos
        self.asignar_trabajos()   # termina antes del siguiente mensaje
        self.preparar_cierre()    # si no queda nada pendiente, se detiene
```

La Figura 2 muestra cómo se procesan dos solicitudes que llegan casi al mismo tiempo cuando solo hay una impresora: ambas esperan en el buzón, el servidor las atiende una por una y el segundo trabajo queda en la cola hasta que la impresora avisa que terminó.

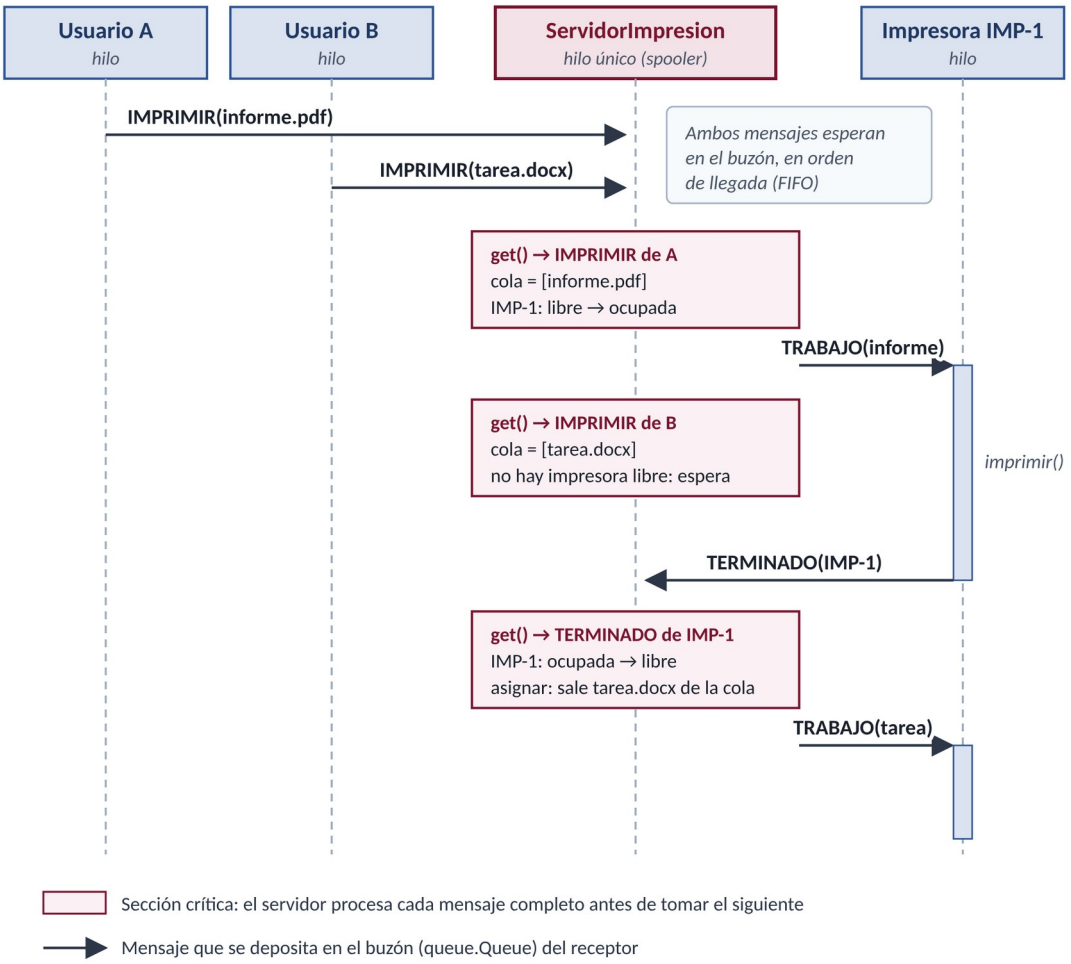


Figura 2 Procesamiento de mensajes en el servidor (sección crítica)

Comparación de las representaciones

La Tabla 4 compara las dos representaciones propuestas para el recurso.

Tabla 4 Comparación de las representaciones del recurso

Criterio	Tipo básico (contadores)	Tipo complejo (diccionario y cola)
Información que guarda	Solo cuántas impresoras están libres y cuántos trabajos esperan.	El estado de cada impresora, el trabajo que imprime y los datos de cada trabajo en espera.
Elección de la impresora	No indica cuál impresora está libre; haría falta otra estructura.	Se busca en el diccionario una impresora con estado "libre".
Orden de atención (FIFO)	No lo registra.	La deque conserva el orden de llegada: se agrega al final y se atiende desde el inicio.
Seguimiento del sistema	No permite saber qué documento está en cada impresora.	Permite mostrar el estado completo y detectar trabajos perdidos o duplicados.

Criterio	Tipo básico (contadores)	Tipo complejo (diccionario y cola)
Simplicidad	Muy simple: solo sumar y restar.	Requiere más código para recorrer y actualizar las estructuras.
Uso de memoria	Mínimo.	Mayor, aunque pequeño para el número de impresoras de una red local.
Riesgo de sincronización en este diseño	Ninguno adicional, porque solo el servidor lo modifica.	Ninguno adicional, por la misma razón.

Representación elegida. Se utilizará la representación de **tipo complejo** (diccionario de impresoras y cola deque de trabajos). El servidor no solo necesita saber cuántas impresoras están libres, sino cuál de ellas recibirá el trabajo, qué documento imprime cada una y en qué orden llegaron las solicitudes, y un contador no guarda esa información. Además, el contador no se pierde, porque se puede calcular a partir del diccionario cuando se necesite:

```
libres = sum(1 for imp in impresoras.values() if imp["estado"] == "libre")
```

Por último, la principal desventaja de una estructura compleja, que es tener que actualizar varios campos sin ser interrumpido, no afecta a este diseño: solo el servidor modifica el recurso y cada mensaje se procesa por completo antes del siguiente.

Bibliografía

Python Software Foundation. (n.d.-a). *collections — Container datatypes*. Retrieved October 29, 2026 from https://docs.python.org/3/library/collections.html

Python Software Foundation. (n.d.-b). *queue — A synchronized queue class*. Retrieved October 29, 2026 from https://docs.python.org/3/library/queue.html

Python Software Foundation. (n.d.-c). *threading — Thread-based parallelism*. Retrieved October 29, 2026 from https://docs.python.org/3/library/threading.html